Module 44

Indexing and Hashing/4: Hashing

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
ppd@cse.iitkgp.ac.in

March 17, 2026

Module Recap

- Understood the design of B+ Tree Index Files in depth for database persistent store

Module Recap

- Understood the design of B+ Tree Index Files in depth for database persistent store
- Familiarized with B-Tree Index Files

Module Objectives

- To explore various hashing schemes - Static and Dynamic Hashing

Module Objectives

- To explore various hashing schemes - Static and Dynamic Hashing
- To compare Ordered Indexing and Hashing

Module Objectives

- To explore various hashing schemes - Static and Dynamic Hashing
- To compare Ordered Indexing and Hashing
- To understand the Bitmap Indices

Module Outline

- Static Hashing

Module Outline

- Static Hashing
- Dynamic Hashing

Module Outline

- Static Hashing
- Dynamic Hashing
- Comparison of Ordered Indexing and Hashing

Module Outline

- Static Hashing
- Dynamic Hashing
- Comparison of Ordered Indexing and Hashing
- Bitmap Indices

Static Hashing

Hash Function

- A hash function h maps data of arbitrary size (from domain D) to fixed-size values (say, integers from 0 to $N > 0$) $h : D \rightarrow [0 \ldots N]$

Hash Function

- A hash function h maps data of arbitrary size (from domain D) to fixed-size values (say, integers from 0 to $N > 0$) $h : D \rightarrow [0 \ldots N]$
- Given key k, $h(k)$ is called hash values, hash codes, digests, or simply hashes

Hash Function

- A hash function h maps data of arbitrary size (from domain D) to fixed-size values (say, integers from 0 to $N > 0$) $h : D \rightarrow [0 \ldots N]$
- Given key k, $h(k)$ is called hash values, hash codes, digests, or simply hashes
- If for two keys $k_1 \neq k_2$, we have $h(k_1) = h(k_2)$, we say a **collision** has occurred

Hash Function

- A hash function h maps data of arbitrary size (from domain D) to fixed-size values (say, integers from 0 to $N > 0$) $h : D \rightarrow [0 \ldots N]$
- Given key k, $h(k)$ is called hash values, hash codes, digests, or simply hashes
- If for two keys $k_1 \neq k_2$, we have $h(k_1) = h(k_2)$, we say a **collision** has occurred
- A hash function should be **Collision Free** and **Fast**

Static Hashing

- A **bucket** is a unit of storage containing one or more records (a bucket is typically a disk block)

Static Hashing

- A **bucket** is a unit of storage containing one or more records (a bucket is typically a disk block)
- In a hash file organization we obtain the bucket of a record directly from its search-key value using a hash function

Static Hashing

- A **bucket** is a unit of storage containing one or more records (a bucket is typically a disk block)
- In a hash file organization we obtain the bucket of a record directly from its search-key value using a hash function
- Hash function h is a function from the set of all search-key values K to the set of all bucket addresses B

Static Hashing

- A **bucket** is a unit of storage containing one or more records (a bucket is typically a disk block)
- In a hash file organization we obtain the bucket of a record directly from its search-key value using a hash function
- Hash function h is a function from the set of all search-key values K to the set of all bucket addresses B
- Hash function is used to locate records for access, insertion as well as deletion

Static Hashing

- A **bucket** is a unit of storage containing one or more records (a bucket is typically a disk block)
- In a hash file organization we obtain the bucket of a record directly from its search-key value using a hash function
- Hash function h is a function from the set of all search-key values K to the set of all bucket addresses B
- Hash function is used to locate records for access, insertion as well as deletion
- Records with different search-key values may be mapped to the same bucket; thus entire bucket has to be searched sequentially to locate a record

Hash Functions

- Worst hash function maps all search-key values to the same bucket; this makes access time proportional to the number of search-key values in the file

Hash Functions

- Worst hash function maps all search-key values to the same bucket; this makes access time proportional to the number of search-key values in the file
- An ideal hash function is **uniform**, i.e., each bucket is assigned the same number of search-key values from the set of all possible values

Hash Functions

- Worst hash function maps all search-key values to the same bucket; this makes access time proportional to the number of search-key values in the file
- An ideal hash function is **uniform**, i.e., each bucket is assigned the same number of search-key values from the set of all possible values
- Ideal hash function is **random**, so each bucket will have the same number of records assigned to it irrespective of the actual distribution of search-key values in the file

Hash Functions

- Worst hash function maps all search-key values to the same bucket; this makes access time proportional to the number of search-key values in the file
- An ideal hash function is **uniform**, i.e., each bucket is assigned the same number of search-key values from the set of all possible values
- Ideal hash function is **random**, so each bucket will have the same number of records assigned to it irrespective of the actual distribution of search-key values in the file
- Typical hash functions perform computation on the internal binary representation of the search-key

Hash Functions

- Worst hash function maps all search-key values to the same bucket; this makes access time proportional to the number of search-key values in the file
- An ideal hash function is **uniform**, i.e., each bucket is assigned the same number of search-key values from the set of all possible values
- Ideal hash function is **random**, so each bucket will have the same number of records assigned to it irrespective of the actual distribution of search-key values in the file
- Typical hash functions perform computation on the internal binary representation of the search-key
- For example, for a string search-key, the binary representations of all the characters in the string could be added and the sum modulo the number of buckets could be returned

Handling of Bucket Overflows

- Bucket overflow can occur because of

Handling of Bucket Overflows

- Bucket overflow can occur because of
  - Insufficient buckets

Handling of Bucket Overflows

- Bucket overflow can occur because of
  - Insufficient buckets
  - Skew in distribution of records. This can occur due to two reasons:

Handling of Bucket Overflows

- Bucket overflow can occur because of
  - Insufficient buckets
  - Skew in distribution of records. This can occur due to two reasons:
    - ▷ multiple records have same search-key value

Handling of Bucket Overflows

- Bucket overflow can occur because of
  - Insufficient buckets
  - Skew in distribution of records. This can occur due to two reasons:
    - ▷ multiple records have same search-key value
    - ▷ chosen hash function produces non-uniform distribution of key values

Handling of Bucket Overflows

- Bucket overflow can occur because of
  - Insufficient buckets
  - Skew in distribution of records. This can occur due to two reasons:
    - ▷ multiple records have same search-key value
    - ▷ chosen hash function produces non-uniform distribution of key values
- Although the probability of bucket overflow can be reduced, it cannot be eliminated

Handling of Bucket Overflows

- Bucket overflow can occur because of
  - Insufficient buckets
  - Skew in distribution of records. This can occur due to two reasons:
    - ▷ multiple records have same search-key value
    - ▷ chosen hash function produces non-uniform distribution of key values
- Although the probability of bucket overflow can be reduced, it cannot be eliminated
- it is handled by using **overflow buckets**

Handling of Bucket Overflows (2)

- **Overflow chaining** - the overflow buckets of a given bucket are chained together in a linked list

Handling of Bucket Overflows (2)

- **Overflow chaining** - the overflow buckets of a given bucket are chained together in a linked list
- Above scheme is called **closed hashing**

Handling of Bucket Overflows (2)

- **Overflow chaining** - the overflow buckets of a given bucket are chained together in a linked list
- Above scheme is called **closed hashing**
- An alternative, called **open hashing**, which does not use overflow buckets, is not suitable for database applications

Hash Indices

- Hashing can be used not only for file organization, but also for index-structure creation

Hash Indices

- Hashing can be used not only for file organization, but also for index-structure creation
- A **hash index** organizes the search keys, with their associated record pointers, into a hash file structure

Hash Indices

- Hashing can be used not only for file organization, but also for index-structure creation
- A **hash index** organizes the search keys, with their associated record pointers, into a hash file structure
- Strictly speaking, hash indices are always secondary indices

Hash Indices

- Hashing can be used not only for file organization, but also for index-structure creation
- A **hash index** organizes the search keys, with their associated record pointers, into a hash file structure
- Strictly speaking, hash indices are always secondary indices
- if the file itself is organized using hashing, a separate primary hash index on it using the same search-key is unnecessary

Hash Indices

- Hashing can be used not only for file organization, but also for index-structure creation
- A **hash index** organizes the search keys, with their associated record pointers, into a hash file structure
- Strictly speaking, hash indices are always secondary indices
- if the file itself is organized using hashing, a separate primary hash index on it using the same search-key is unnecessary
- However, we use the term hash index to refer to both secondary index structures and hash organized files

Deficiencies of Static Hashing

Important

- In static hashing, function h maps search-key values to a fixed set of B of bucket addresses. Databases grow or shrink with time

Deficiencies of Static Hashing

Important

- In static hashing, function h maps search-key values to a fixed set of B of bucket addresses. Databases grow or shrink with time
- If initial number of buckets is too small, and file grows, performance will degrade due to too much overflows

Deficiencies of Static Hashing

Important

- In static hashing, function h maps search-key values to a fixed set of B of bucket addresses. Databases grow or shrink with time
- If initial number of buckets is too small, and file grows, performance will degrade due to too much overflows
- If space is allocated for anticipated growth, a significant amount of space will be wasted initially (and buckets will be underfull).

Deficiencies of Static Hashing

Important

- In static hashing, function h maps search-key values to a fixed set of B of bucket addresses. Databases grow or shrink with time
- If initial number of buckets is too small, and file grows, performance will degrade due to too much overflows
- If space is allocated for anticipated growth, a significant amount of space will be wasted initially (and buckets will be underfull).
- If database shrinks, again space will be wasted

Deficiencies of Static Hashing

Important

- In static hashing, function h maps search-key values to a fixed set of B of bucket addresses. Databases grow or shrink with time
- If initial number of buckets is too small, and file grows, performance will degrade due to too much overflows
- If space is allocated for anticipated growth, a significant amount of space will be wasted initially (and buckets will be underfull).
- If database shrinks, again space will be wasted
- One solution: periodic re-organization of the file with a new hash function (Expensive, disrupts normal operations)

Deficiencies of Static Hashing

Important

- In static hashing, function h maps search-key values to a fixed set of B of bucket addresses. Databases grow or shrink with time
- If initial number of buckets is too small, and file grows, performance will degrade due to too much overflows
- If space is allocated for anticipated growth, a significant amount of space will be wasted initially (and buckets will be underfull).
- If database shrinks, again space will be wasted
- One solution: periodic re-organization of the file with a new hash function (Expensive, disrupts normal operations)
- Better solution: allow the number of buckets to be modified dynamically

Dynamic Hashing

Dynamic Hashing

- Good for database that grows and shrinks in size

Dynamic Hashing

- Good for database that grows and shrinks in size
- Allows the hash function to be modified dynamically

Dynamic Hashing

- Good for database that grows and shrinks in size
- Allows the hash function to be modified dynamically
- **Extendable hashing** - one form of dynamic hashing

Dynamic Hashing

- Good for database that grows and shrinks in size
- Allows the hash function to be modified dynamically
- **Extendable hashing** - one form of dynamic hashing
- Hash function generates values over a large range - typically b-bit integers, with $b = 32$

Dynamic Hashing

- Good for database that grows and shrinks in size
- Allows the hash function to be modified dynamically
- **Extendable hashing** - one form of dynamic hashing
- Hash function generates values over a large range - typically b-bit integers, with $b = 32$
- At any time use only a prefix of the hash function to index into a table of bucket addresses

Dynamic Hashing

- Good for database that grows and shrinks in size
- Allows the hash function to be modified dynamically
- **Extendable hashing** - one form of dynamic hashing
- Hash function generates values over a large range - typically b-bit integers, with $b = 32$
- At any time use only a prefix of the hash function to index into a table of bucket addresses
- Let the length of the prefix be i bits, $0 \leq i \leq 32$

Dynamic Hashing

- Good for database that grows and shrinks in size
- Allows the hash function to be modified dynamically
- **Extendable hashing** - one form of dynamic hashing
- Hash function generates values over a large range - typically b-bit integers, with $b = 32$
- At any time use only a prefix of the hash function to index into a table of bucket addresses
- Let the length of the prefix be i bits, $0 \leq i \leq 32$
  - ▷ Bucket address table size $= 2^i$. Initially $i = 0$

Dynamic Hashing

- Good for database that grows and shrinks in size
- Allows the hash function to be modified dynamically
- **Extendable hashing** - one form of dynamic hashing
- Hash function generates values over a large range - typically b-bit integers, with $b = 32$
- At any time use only a prefix of the hash function to index into a table of bucket addresses
- Let the length of the prefix be i bits, $0 \leq i \leq 32$
  - ▷ Bucket address table size $= 2^i$. Initially $i = 0$
  - ▷ Value of i grows and shrinks as the size of the database grows and shrinks

Dynamic Hashing

- Good for database that grows and shrinks in size
- Allows the hash function to be modified dynamically
- **Extendable hashing** - one form of dynamic hashing
- Hash function generates values over a large range - typically b-bit integers, with $b = 32$
- At any time use only a prefix of the hash function to index into a table of bucket addresses
- Let the length of the prefix be i bits, $0 \leq i \leq 32$
  - ▷ Bucket address table size $= 2^i$. Initially $i = 0$
  - ▷ Value of i grows and shrinks as the size of the database grows and shrinks
- Multiple entries in the bucket address table may point to a bucket

Dynamic Hashing

- Good for database that grows and shrinks in size
- Allows the hash function to be modified dynamically
- **Extendable hashing** - one form of dynamic hashing
- Hash function generates values over a large range - typically b-bit integers, with $b = 32$
- At any time use only a prefix of the hash function to index into a table of bucket addresses
- Let the length of the prefix be i bits, $0 \leq i \leq 32$
  - ▷ Bucket address table size $= 2^i$. Initially $i = 0$
  - ▷ Value of i grows and shrinks as the size of the database grows and shrinks
- Multiple entries in the bucket address table may point to a bucket
- Thus, actual number of buckets is $< 2^i$ (number of buckets changes dynamically due to coalescing and splitting of buckets)

Use of Extendable Hash Structure

- Each bucket j stores a value i_j

Use of Extendable Hash Structure

- Each bucket j stores a value i_j
- All the entries that point to the same bucket have the same values on the first i_j bits

Use of Extendable Hash Structure

- Each bucket j stores a value i_j
- All the entries that point to the same bucket have the same values on the first i_j bits
- To locate the bucket containing search-key K_j

Use of Extendable Hash Structure

- Each bucket j stores a value i_j
- All the entries that point to the same bucket have the same values on the first i_j bits
- To locate the bucket containing search-key K_j
  - Compute $h(K_j) = X$

Use of Extendable Hash Structure

- Each bucket j stores a value i_j
- All the entries that point to the same bucket have the same values on the first i_j bits
- To locate the bucket containing search-key K_j
  - Compute $h(K_j) = X$
  - Use the first i high order bits of X as a displacement into bucket address table, and follow the pointer to appropriate bucket

Use of Extendable Hash Structure

- Each bucket j stores a value i_j
- All the entries that point to the same bucket have the same values on the first i_j bits
- To locate the bucket containing search-key K_j
  - Compute $h(K_j) = X$
  - Use the first i high order bits of X as a displacement into bucket address table, and follow the pointer to appropriate bucket
- To insert a record with search-key value K_j

Use of Extendable Hash Structure

- Each bucket j stores a value i_j
- All the entries that point to the same bucket have the same values on the first i_j bits
- To locate the bucket containing search-key K_j
  - Compute $h(K_j) = X$
  - Use the first i high order bits of X as a displacement into bucket address table, and follow the pointer to appropriate bucket
- To insert a record with search-key value K_j
  - Follow same procedure as look-up and locate the bucket, say j

Use of Extendable Hash Structure

- Each bucket j stores a value i_j
- All the entries that point to the same bucket have the same values on the first i_j bits
- To locate the bucket containing search-key K_j
  - Compute $h(K_j) = X$
  - Use the first i high order bits of X as a displacement into bucket address table, and follow the pointer to appropriate bucket
- To insert a record with search-key value K_j
  - Follow same procedure as look-up and locate the bucket, say j
  - If there is room in the bucket j insert record in the bucket

Use of Extendable Hash Structure

- Each bucket j stores a value i_j
- All the entries that point to the same bucket have the same values on the first i_j bits
- To locate the bucket containing search-key K_j
  - Compute $h(K_j) = X$
  - Use the first i high order bits of X as a displacement into bucket address table, and follow the pointer to appropriate bucket
- To insert a record with search-key value K_j
  - Follow same procedure as look-up and locate the bucket, say j
  - If there is room in the bucket j insert record in the bucket
  - Else the bucket must be split and insertion re-attempted

Use of Extendable Hash Structure

- Each bucket j stores a value i_j
- All the entries that point to the same bucket have the same values on the first i_j bits
- To locate the bucket containing search-key K_j
  - Compute $h(K_j) = X$
  - Use the first i high order bits of X as a displacement into bucket address table, and follow the pointer to appropriate bucket
- To insert a record with search-key value K_j
  - Follow same procedure as look-up and locate the bucket, say j
  - If there is room in the bucket j insert record in the bucket
  - Else the bucket must be split and insertion re-attempted
  - ▷ Overflow buckets used instead in some cases

Insertion in Extendable Hash Structure

- If $i > i_j$ (more than one pointer to bucket j)

Insertion in Extendable Hash Structure

- If $i > i_j$ (more than one pointer to bucket j)
  - Allocate a new bucket z, and set $i_j = i_z = (i_j + 1)$

Insertion in Extendable Hash Structure

- If $i > i_j$ (more than one pointer to bucket j)
  - Allocate a new bucket z, and set $i_j = i_z = (i_j + 1)$
  - Update the second half of the bucket address table entries originally pointing to j, to point to z

Insertion in Extendable Hash Structure

- If $i > i_j$ (more than one pointer to bucket j)
  - Allocate a new bucket z, and set $i_j = i_z = (i_j + 1)$
  - Update the second half of the bucket address table entries originally pointing to j, to point to z
  - Remove each record in bucket j and reinsert (in j or z)

Insertion in Extendable Hash Structure

- If $i > i_j$ (more than one pointer to bucket j)
  - Allocate a new bucket z, and set $i_j = i_z = (i_j + 1)$
  - Update the second half of the bucket address table entries originally pointing to j, to point to z
  - Remove each record in bucket j and reinsert (in j or z)
  - Recompute new bucket for K_j and insert record in the bucket (further splitting is required if the bucket is still full)

Insertion in Extendable Hash Structure

- If $i > i_j$ (more than one pointer to bucket j)
  - Allocate a new bucket z, and set $i_j = i_z = (i_j + 1)$
  - Update the second half of the bucket address table entries originally pointing to j, to point to z
  - Remove each record in bucket j and reinsert (in j or z)
  - Recompute new bucket for K_j and insert record in the bucket (further splitting is required if the bucket is still full)
- If $i = i_j$ (only one pointer to bucket j)

Insertion in Extendable Hash Structure

- If $i > i_j$ (more than one pointer to bucket j)
  - Allocate a new bucket z, and set $i_j = i_z = (i_j + 1)$
  - Update the second half of the bucket address table entries originally pointing to j, to point to z
  - Remove each record in bucket j and reinsert (in j or z)
  - Recompute new bucket for K_j and insert record in the bucket (further splitting is required if the bucket is still full)
- If $i = i_j$ (only one pointer to bucket j)
  - If i reaches some limit b, or too many splits have happened in this insertion, create an overflow bucket

Insertion in Extendable Hash Structure

- If $i > i_j$ (more than one pointer to bucket j)
  - Allocate a new bucket z, and set $i_j = i_z = (i_j + 1)$
  - Update the second half of the bucket address table entries originally pointing to j, to point to z
  - Remove each record in bucket j and reinsert (in j or z)
  - Recompute new bucket for K_j and insert record in the bucket (further splitting is required if the bucket is still full)
- If $i = i_j$ (only one pointer to bucket j)
  - If i reaches some limit b, or too many splits have happened in this insertion, create an overflow bucket
  - Else: Increment i and double the size of the bucket address table. Replace each entry in the table by two entries that point to the same bucket. Recompute new bucket address table entry for K_j. Now $i > i_j$ so use the first case above.

Deletion in Extendable Hash Structure

- To delete a key value:

Deletion in Extendable Hash Structure

- To delete a key value:
  - locate it in its bucket and remove it

Deletion in Extendable Hash Structure

- To delete a key value:
  - locate it in its bucket and remove it
  - The bucket itself can be removed if it becomes empty (with appropriate updates to the bucket address table)

Deletion in Extendable Hash Structure

- To delete a key value:
  - locate it in its bucket and remove it
  - The bucket itself can be removed if it becomes empty (with appropriate updates to the bucket address table)
- Coalescing of buckets can be done (can coalesce only with a 'buddy' bucket having same value of i_j and same i_{j-1} prefix, if it is present)

Deletion in Extendable Hash Structure

- To delete a key value:
  - locate it in its bucket and remove it
  - The bucket itself can be removed if it becomes empty (with appropriate updates to the bucket address table)
- Coalescing of buckets can be done (can coalesce only with a 'buddy' bucket having same value of i_j and same i_{j-1} prefix, if it is present)
- Decreasing bucket address table size is also possible

Deletion in Extendable Hash Structure

- To delete a key value:
  - locate it in its bucket and remove it
  - The bucket itself can be removed if it becomes empty (with appropriate updates to the bucket address table)
- Coalescing of buckets can be done (can coalesce only with a 'buddy' bucket having same value of i_j and same i_{j-1} prefix, if it is present)
- Decreasing bucket address table size is also possible
- ▷ Note: decreasing bucket address table size is an expensive operation and should be done only if number of buckets becomes much smaller than the size of the table

Comparison Schemes

Extendable Hashing vs. Other Schemes

- **Benefits of extendable hashing**:

Extendable Hashing vs. Other Schemes

- **Benefits of extendable hashing**:
  - Hash performance does not degrade with growth of file

Extendable Hashing vs. Other Schemes

- **Benefits of extendable hashing**:
  - Hash performance does not degrade with growth of file
  - Minimal space overhead

Extendable Hashing vs. Other Schemes

- **Benefits of extendable hashing**:
  - Hash performance does not degrade with growth of file
  - Minimal space overhead
- **Disadvantages of extendable hashing**

Extendable Hashing vs. Other Schemes

- **Benefits of extendable hashing**:
  - Hash performance does not degrade with growth of file
  - Minimal space overhead
- **Disadvantages of extendable hashing**
  - Extra level of indirection to find desired record

Extendable Hashing vs. Other Schemes

- **Benefits of extendable hashing**:
  - Hash performance does not degrade with growth of file
  - Minimal space overhead
- **Disadvantages of extendable hashing**
  - Extra level of indirection to find desired record
  - Bucket address table may itself become very big (larger than memory)

Extendable Hashing vs. Other Schemes

- **Benefits of extendable hashing**:
  - Hash performance does not degrade with growth of file
  - Minimal space overhead
- **Disadvantages of extendable hashing**
  - Extra level of indirection to find desired record
  - Bucket address table may itself become very big (larger than memory)
  - ▷ Cannot allocate very large contiguous areas on disk either

Extendable Hashing vs. Other Schemes

- **Benefits of extendable hashing**:
  - Hash performance does not degrade with growth of file
  - Minimal space overhead
- **Disadvantages of extendable hashing**
  - Extra level of indirection to find desired record
  - Bucket address table may itself become very big (larger than memory)
  - ▷ Cannot allocate very large contiguous areas on disk either
  - ▷ Solution: B+ -tree structure to locate desired record in bucket address table

Extendable Hashing vs. Other Schemes

- **Benefits of extendable hashing**:
  - Hash performance does not degrade with growth of file
  - Minimal space overhead
- **Disadvantages of extendable hashing**
  - Extra level of indirection to find desired record
  - Bucket address table may itself become very big (larger than memory)
  - ▷ Cannot allocate very large contiguous areas on disk either
  - ▷ Solution: B+ -tree structure to locate desired record in bucket address table
  - Changing size of bucket address table is an expensive operation

Extendable Hashing vs. Other Schemes

- **Benefits of extendable hashing**:
  - Hash performance does not degrade with growth of file
  - Minimal space overhead
- **Disadvantages of extendable hashing**
  - Extra level of indirection to find desired record
  - Bucket address table may itself become very big (larger than memory)
  - ▷ Cannot allocate very large contiguous areas on disk either
  - ▷ Solution: B+ -tree structure to locate desired record in bucket address table
  - Changing size of bucket address table is an expensive operation
- **Linear hashing is an alternative mechanism**

Extendable Hashing vs. Other Schemes

- **Benefits of extendable hashing**:
  - Hash performance does not degrade with growth of file
  - Minimal space overhead
- **Disadvantages of extendable hashing**
  - Extra level of indirection to find desired record
  - Bucket address table may itself become very big (larger than memory)
  - ▷ Cannot allocate very large contiguous areas on disk either
  - ▷ Solution: B+ -tree structure to locate desired record in bucket address table
  - Changing size of bucket address table is an expensive operation
- **Linear hashing is an alternative mechanism**
  - Allows incremental growth of its directory (equivalent to bucket address table)

Extendable Hashing vs. Other Schemes

- **Benefits of extendable hashing**:
  - Hash performance does not degrade with growth of file
  - Minimal space overhead
- **Disadvantages of extendable hashing**
  - Extra level of indirection to find desired record
  - Bucket address table may itself become very big (larger than memory)
  - ▷ Cannot allocate very large contiguous areas on disk either
  - ▷ Solution: B+ -tree structure to locate desired record in bucket address table
  - Changing size of bucket address table is an expensive operation
- **Linear hashing is an alternative mechanism**
  - Allows incremental growth of its directory (equivalent to bucket address table)
  - At the cost of more bucket overflows

Comparison of Ordered Indexing and Hashing

- Cost of periodic re-organization

Comparison of Ordered Indexing and Hashing

- Cost of periodic re-organization
- Relative frequency of insertions and deletions

Comparison of Ordered Indexing and Hashing

- Cost of periodic re-organization
- Relative frequency of insertions and deletions
- Is it desirable to optimize average access time at the expense of worst-case access time?

Comparison of Ordered Indexing and Hashing

- Cost of periodic re-organization
- Relative frequency of insertions and deletions
- Is it desirable to optimize average access time at the expense of worst-case access time?
- Expected type of queries:

Comparison of Ordered Indexing and Hashing

- Cost of periodic re-organization
- Relative frequency of insertions and deletions
- Is it desirable to optimize average access time at the expense of worst-case access time?
- Expected type of queries:
  - Hashing is generally better at retrieving records having a specified value of the key

Comparison of Ordered Indexing and Hashing

- Cost of periodic re-organization
- Relative frequency of insertions and deletions
- Is it desirable to optimize average access time at the expense of worst-case access time?
- Expected type of queries:
  - Hashing is generally better at retrieving records having a specified value of the key
  - If range queries are common, ordered indices are to be preferred

Comparison of Ordered Indexing and Hashing

- Cost of periodic re-organization
- Relative frequency of insertions and deletions
- Is it desirable to optimize average access time at the expense of worst-case access time?
- Expected type of queries:
  - Hashing is generally better at retrieving records having a specified value of the key
  - If range queries are common, ordered indices are to be preferred
- In practice:

Comparison of Ordered Indexing and Hashing

- Cost of periodic re-organization
- Relative frequency of insertions and deletions
- Is it desirable to optimize average access time at the expense of worst-case access time?
- Expected type of queries:
  - Hashing is generally better at retrieving records having a specified value of the key
  - If range queries are common, ordered indices are to be preferred
- In practice:
  - PostgreSQL supports hash indices, but discourages use due to poor performance

Comparison of Ordered Indexing and Hashing

- Cost of periodic re-organization
- Relative frequency of insertions and deletions
- Is it desirable to optimize average access time at the expense of worst-case access time?
- Expected type of queries:
  - Hashing is generally better at retrieving records having a specified value of the key
  - If range queries are common, ordered indices are to be preferred
- In practice:
  - PostgreSQL supports hash indices, but discourages use due to poor performance
  - Oracle supports static hash organization, but not hash indices

Comparison of Ordered Indexing and Hashing

- Cost of periodic re-organization
- Relative frequency of insertions and deletions
- Is it desirable to optimize average access time at the expense of worst-case access time?
- Expected type of queries:
  - Hashing is generally better at retrieving records having a specified value of the key
  - If range queries are common, ordered indices are to be preferred
- In practice:
  - PostgreSQL supports hash indices, but discourages use due to poor performance
  - Oracle supports static hash organization, but not hash indices
  - SQLServer supports only B+ -trees

Bitmap Indices

Bitmap Indices

- Bitmap indices are a special type of index designed for efficient querying on multiple keys

Bitmap Indices

- Bitmap indices are a special type of index designed for efficient querying on multiple keys
- Records in a relation are assumed to be numbered sequentially from, say, 0

Bitmap Indices

- Bitmap indices are a special type of index designed for efficient querying on multiple keys
- Records in a relation are assumed to be numbered sequentially from, say, 0
- Given a number n it must be easy to retrieve record n

Bitmap Indices

- Bitmap indices are a special type of index designed for efficient querying on multiple keys
- Records in a relation are assumed to be numbered sequentially from, say, 0
- Given a number n it must be easy to retrieve record n
- ▷ Particularly easy if records are of fixed size

Bitmap Indices

- Bitmap indices are a special type of index designed for efficient querying on multiple keys
- Records in a relation are assumed to be numbered sequentially from, say, 0
- Given a number n it must be easy to retrieve record n
- ▷ Particularly easy if records are of fixed size
- Applicable on attributes that take on a relatively small number of distinct values

Bitmap Indices

- Bitmap indices are a special type of index designed for efficient querying on multiple keys
- Records in a relation are assumed to be numbered sequentially from, say, 0
- Given a number n it must be easy to retrieve record n
- ▷ Particularly easy if records are of fixed size
- Applicable on attributes that take on a relatively small number of distinct values
  - For example: gender, country, state, . . .

Bitmap Indices

- Bitmap indices are a special type of index designed for efficient querying on multiple keys
- Records in a relation are assumed to be numbered sequentially from, say, 0
- Given a number n it must be easy to retrieve record n
- ▷ Particularly easy if records are of fixed size
- Applicable on attributes that take on a relatively small number of distinct values
  - For example: gender, country, state, ...
  - For example: income-level (income broken up into a small number of levels such as 0-9999, 10000-19999, 20000-50000, 50000-infinity)

Bitmap Indices

- Bitmap indices are a special type of index designed for efficient querying on multiple keys
- Records in a relation are assumed to be numbered sequentially from, say, 0
- Given a number n it must be easy to retrieve record n
- ▷ Particularly easy if records are of fixed size
- Applicable on attributes that take on a relatively small number of distinct values
  - For example: gender, country, state, ...
  - For example: income-level (income broken up into a small number of levels such as 0-9999, 10000-19999, 20000-50000, 50000-infinity)
- A bitmap is simply an array of bits

Bitmap Indices (2)

- In its simplest form a bitmap index on an attribute has a bitmap for each value of the attribute

Bitmap Indices (2)

- In its simplest form a bitmap index on an attribute has a bitmap for each value of the attribute
- Bitmap has as many bits as records

Bitmap Indices (2)

- In its simplest form a bitmap index on an attribute has a bitmap for each value of the attribute
- Bitmap has as many bits as records
- In a bitmap for value v, the bit for a record is 1 if the record has the value v for the attribute, and is 0 otherwise

Bitmap Indices (3)

- Bitmap indices are useful for queries on multiple attributes

Bitmap Indices (3)

- Bitmap indices are useful for queries on multiple attributes
- not particularly useful for single attribute queries

Bitmap Indices (3)

- Bitmap indices are useful for queries on multiple attributes
- not particularly useful for single attribute queries
- Queries are answered using bitmap operations

Bitmap Indices (3)

- Bitmap indices are useful for queries on multiple attributes
- not particularly useful for single attribute queries
- Queries are answered using bitmap operations
    - Intersection (and)

Bitmap Indices (3)

- Bitmap indices are useful for queries on multiple attributes
- not particularly useful for single attribute queries
- Queries are answered using bitmap operations
  - Intersection (and)
  - Union (or)

Bitmap Indices (3)

- Bitmap indices are useful for queries on multiple attributes
- not particularly useful for single attribute queries
- Queries are answered using bitmap operations
  - Intersection (and)
  - Union (or)
  - Complementation (not)

Bitmap Indices (3)

- Bitmap indices are useful for queries on multiple attributes
- not particularly useful for single attribute queries
- Queries are answered using bitmap operations
  - Intersection (and)
  - Union (or)
  - Complementation (not)
- Each operation takes two bitmaps of the same size and applies the operation on corresponding bits to get the result bitmap

Bitmap Indices (3)

- Bitmap indices are useful for queries on multiple attributes
- not particularly useful for single attribute queries
- Queries are answered using bitmap operations
  - Intersection (and)
  - Union (or)
  - Complementation (not)
- Each operation takes two bitmaps of the same size and applies the operation on corresponding bits to get the result bitmap
- For example: 100110 AND 110011 = 100010, 100110 OR 110011 = 110111, NOT 100110 = 011001

Bitmap Indices (3)

- Bitmap indices are useful for queries on multiple attributes
- not particularly useful for single attribute queries
- Queries are answered using bitmap operations
  - Intersection (and)
  - Union (or)
  - Complementation (not)
- Each operation takes two bitmaps of the same size and applies the operation on corresponding bits to get the result bitmap
- For example: 100110 AND 110011 = 100010, 100110 OR 110011 = 110111, NOT 100110 = 011001
- Males with income level L1: 10010 AND 10100 = 10000

Bitmap Indices (3)

- Bitmap indices are useful for queries on multiple attributes
- not particularly useful for single attribute queries
- Queries are answered using bitmap operations
  - Intersection (and)
  - Union (or)
  - Complementation (not)
- Each operation takes two bitmaps of the same size and applies the operation on corresponding bits to get the result bitmap
- For example: 100110 AND 110011 = 100010, 100110 OR 110011 = 110111, NOT 100110 = 011001
- Males with income level L1: 10010 AND 10100 = 10000
  - ▷ Can then retrieve required tuples

Bitmap Indices (3)

- Bitmap indices are useful for queries on multiple attributes
- not particularly useful for single attribute queries
- Queries are answered using bitmap operations
  - Intersection (and)
  - Union (or)
  - Complementation (not)
- Each operation takes two bitmaps of the same size and applies the operation on corresponding bits to get the result bitmap
- For example: 100110 AND 110011 = 100010, 100110 OR 110011 = 110111, NOT 100110 = 011001
- Males with income level L1: 10010 AND 10100 = 10000
  - ▷ Can then retrieve required tuples
  - ▷ Counting number of matching tuples is even faster

Bitmap Indices (4)

- Bitmap indices generally very small compared with relation size

Bitmap Indices (4)

- Bitmap indices generally very small compared with relation size
- For example, if record is 100 bytes, space for a single bitmap is 1/800 of space used by relation

Bitmap Indices (4)

- Bitmap indices generally very small compared with relation size
- For example, if record is 100 bytes, space for a single bitmap is 1/800 of space used by relation
- ▷ If number of distinct attribute values is 8, bitmap is only 1% of relation size

Bitmap Indices (4)

- Bitmap indices generally very small compared with relation size
- For example, if record is 100 bytes, space for a single bitmap is 1/800 of space used by relation
- ▷ If number of distinct attribute values is 8, bitmap is only 1% of relation size
- Deletion needs to be handled properly

Bitmap Indices (4)

- Bitmap indices generally very small compared with relation size
- For example, if record is 100 bytes, space for a single bitmap is 1/800 of space used by relation
- ▷ If number of distinct attribute values is 8, bitmap is only 1% of relation size
- Deletion needs to be handled properly
- **Existence bitmap** to note if there is a valid record at a record location (Needed for complementation)

Bitmap Indices (4)

- Bitmap indices generally very small compared with relation size
- For example, if record is 100 bytes, space for a single bitmap is 1/800 of space used by relation
- ▷ If number of distinct attribute values is 8, bitmap is only 1% of relation size
- Deletion needs to be handled properly
- **Existence bitmap** to note if there is a valid record at a record location (Needed for complementation)
- ▷ not($A = v$): (NOT bitmap-A-v) AND ExistenceBitmap

Bitmap Indices (4)

- Bitmap indices generally very small compared with relation size
- For example, if record is 100 bytes, space for a single bitmap is 1/800 of space used by relation
- ▷ If number of distinct attribute values is 8, bitmap is only 1% of relation size
- Deletion needs to be handled properly
- **Existence bitmap** to note if there is a valid record at a record location (Needed for complementation)
- ▷ not($A = v$): (NOT bitmap-A-v) AND ExistenceBitmap
- Should keep bitmaps for all values, even null value

Bitmap Indices (4)

- Bitmap indices generally very small compared with relation size
- For example, if record is 100 bytes, space for a single bitmap is 1/800 of space used by relation
- ▷ If number of distinct attribute values is 8, bitmap is only 1% of relation size
- Deletion needs to be handled properly
- **Existence bitmap** to note if there is a valid record at a record location (Needed for complementation)
- ▷ not($A = v$): (NOT bitmap-A-v) AND ExistenceBitmap
- Should keep bitmaps for all values, even null value
- To correctly handle SQL null semantics for NOT($A = v$):

Bitmap Indices (4)

- Bitmap indices generally very small compared with relation size
- For example, if record is 100 bytes, space for a single bitmap is 1/800 of space used by relation
- ▷ If number of distinct attribute values is 8, bitmap is only 1% of relation size
- Deletion needs to be handled properly
- **Existence bitmap** to note if there is a valid record at a record location (Needed for complementation)
- ▷ not($A = v$): (NOT bitmap-A-v) AND ExistenceBitmap
- Should keep bitmaps for all values, even null value
- To correctly handle SQL null semantics for NOT($A = v$):
- ▷ intersect above result with (NOT bitmap-A-Null)

Module Summary

- Explored various hashing schemes - Static and Dynamic Hashing

Slides used in this presentation are borrowed from `http://db-book.com/` with kind permission of the authors.
Edited and new slides are marked with 'PPD'.

Module Summary

- Explored various hashing schemes - Static and Dynamic Hashing
- Compared Ordered Indexing and Hashing

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

- Explored various hashing schemes - Static and Dynamic Hashing
- Compared Ordered Indexing and Hashing
- Understood Bitmap Indices

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.